CLOUD COMPUTING

Assignment 2 – Research Report

13 JUNE

JOERI HAREMAN
HZ University of Applied Sciences

Contents

1. Results 2
1.2 Services Selection 2
1.3 EC2 3
1.3.1 Services Configuration 3
1.3.2 Installation 3
1.3.3 Implementation 3
1.4 Lambda 4
1.4.1 Services Configuration 4
1.4.2 Installation & setup 4
1.4.3 Implementation 4
1.5 Performance table 5
1.6 Non-functional qualities 5
Conclusion 6
PoC (Appendix A) 7
References 12

1. Results

In this chapter we will go over the outcomes of the research activities described in the methodological section. Each sub-question will be addressed using the AWS services, we will make use of benchmarks, observation, and evaluations. The results will be shown using tables, images, screenshots, descriptive text, and coding where it's needed. The screenshots mentioned in the text are show as (e.g. "as shown in figure"), you are able to right click on the text that says "figure", and you will be taken to the figure itself in the appendix. This structure will be used to eventually answer the main research question.

1.2 Services Selection

Which AWS compute services are most suitable for running large-scale WFA, and what are the trade-offs in terms of scalability and performance?

The first research question askes which AWS services is most suitable for large-scale WFA. I will use different methods to answer this such as, benchmarking execution time and resource consumption, cost logging via CloudWatch, and monitoring nonfunctional qualities. To meet these methods, I have chosen two service categories to test this on:

- EC2, is for traditional batch processing.
- AWS lambda is serverless and works with streaming workloads.

To establish a fair representation of the two services, I decided to create three datasets. These datasets contain small, medium, and large amount of data, to see which service can handle the most amount of data, and to see which service can process the quickest.

- Small dataset = dataset_small.txt (1kb)
- Medium dataset = dataset_medium.txt (~120MB)
- Large dataset = dataset_large.txt (~480MB)

The small dataset is created by repeating the same words, this is done to show the number of times certain words are appearing in the dataset using the WFA. Then the medium dataset is created by downloading public books from project Gutenberg (*Project Gutenberg*, n.d.), to represent a fair WFA. Lastly, the large dataset is created by repeating the medium dataset a few times. I have used these datasets in all of the service tests, to keep a fair representation. To see how I have done this in my control panel using WSL see Figure 1 and Figure 2.

1.3 EC2

1.3.1 Services Configuration

To coordinate the processing pipeline, I designed the following UML diagram. To see the UML component diagram see Figure 3, UML component diagram for .

Diagram explained:

- S3 -> EC2
  - The python script on EC2 streams each dataset from the bucket in 8KB chunks using the S3 GetObject API
- SSH & SCP client -> EC2
  - From my WSL terminal I used the ssh -i wfa-key.pem ubuntu@<EC2_IP> command to connect. And the scp -i ... wfa.py dataset_small.txt ubuntu@...:/home/ubuntu/ to upload code and data.
- AWS CLI -> S3
  - On the EC2 I ran, aws s3 mb, aws s3 cp, and aws s3 ls, to create the bucket and paste the upload the dataset for small, medium, and large.
- File System-> AWS CLI
  - On EC2 I configured AWS CLI through the "aws configure" command, sourcing credentials stored in my local filesystem under ~/.aws/credentials.

1.3.2 Installation

to evaluate the performance of the services, I starter with a benchmark using the EC2 instance. I used the free tier that AWS offers which is the t2.micro instance, and let it run on ubuntu 24.04 LTS. First I will show prove that EC2 is correctly installed on ubuntu on my laptop and connected with my EC2, see Figure 4, Showing that EC2 is correctly installed with ubuntu and .

1.3.3 Implementation

I created a word frequency analysis to check the word count, and which words occur the most in the given dataset, as shown in Figure 5. Then we will execute the code by letting it check the small text file; the results show the most frequent words in that file containing the count. It also showed that even a small t2.micro used from EC2 can handle basic workloads effectively. This is proven in Figure 6. Lastly, I ran both the medium and large dataset to check for not only the word counts but also the processing time. As you can see in the image the word counts are shown, apart from the large dataset, this is not only due to the usage of my free trial, but also due to the lack of sufficient RAM (1GB). Despite this error, it shows the limitations of running large text analysis tasks on low resource virtual machines. This demonstrates that EC2 is sufficient for small datasets, but that larger datasets require scalable computing services, to avoid memory bottlenecks. To show the outputs of the medium and large dataset I have implemented a screenshot of the out put as shown in Figure 7.

1.4 Lambda

1.4.1 Services Configuration

To coordinate the processing pipeline, I designed the following UML diagram. To see the UML component diagram see Figure 8, Lamda UML component .

- S3 → Lambda
  - Streams each dataset in 8 KB chunks.
- Lambda → IAM Role
  - Acquires s3:GetObject permissions via the LambdaS3ReadWFA role.
- Lambda → CloudWatch Logs
  - Emits INFO-level logs (start, progress, results).
- Lambda → CloudWatch Metrics
  - AWS automatically records duration, billed duration, and memory.

1.4.2 Installation & setup

For AWS Lambda we have to setup a few things first, the IAM role, Function creation, General configuration, and code creation.

- IAM Role
  - I created a role LambdaS3ReadWFA, with a managed policy AWSLambdaBasicExecutionRole and an inline policy granting s3:GetObject on arn:aws:s3:::wfa-bucket-joeri-202506/input/*, as shown Figure 9.
- Function Creation
  - For the function creation you go to Lambda -> create function.
    I gave the name `wfa-lambda-stream`, and runtime Python 3.9. Then I inserted the role we created LambdaS3ReadWFA.
- General configuration
  - I set an initial Memory of 512MB and a timeout of 60s. I updated this every time the dataset grew, up to 2048MB and a timeout of 300s. This ensures we had enough CPU and I/O bandwidth to complete the job without premature termination. Proof of this is shown in Figure 10.
- Code creation
  - To check the WFA, we need to create a piece of code that executes the WFA. We let the WFA show the most common words, how many times they appear and the execution time. The code snippet is shown in Figure 11.

1.4.3 Implementation

Then we ran the test for the small dataset, after you execute your test on Lambda you have the option to see the succeeded logs, Figure 12. Here it shows the execution time (63ms), the memory used, and resources configured. The most common word in this small dataset was “hello”. We can conclude that lambda handles trivial workloads immediately, with minimal resources.

For the medium dataset, processing time increased, but it is still under 10 seconds which is quite fast. This demonstrates that the streaming approach avoids memory errors and delivers word-counting events even on 100MB files, this is seen in Figure 13.

Last but not least, we did the test on the large dataset. You see that the execution time has significantly increased, and the max memory used also. The most common used words are still given and the a amount of unique words, results are shown in Figure 14.

To conclude AWS lambda with streaming is suited for small WFA analysis tasks. It completes small tasks around 100MB within seconds, and this with only using the free tier. Even the bigger WFA of 500MB is succeeding in contrary to EC2. However, something you notice is that when the WFA become bigger, the execution time becomes drastically bigger. On a local set-up as mine, it already takes quite some time, for just 500MB.

1.5 Performance table

After doing test on both EC2 and Lambda I will put them against each other to see which one is performing the best.

Metric	EC2 small	EC2 medium	EC2 large	Lambda small	Lambda medium	Lambda large
Execution time (S)	0.2	4.1	N/A	0.063	7.4	28.6
Memory used (MB)	~200	~850	N/A	35	500	1800
Status	success	success	failure	success	success	success

1.6 Non-functional qualities

Apart from the metrics used there are some critical non-functional factors that we need to consider. The EC2 requires you to do a manual provision, SSH set-up, and cleanup, this can increase the operational overhead by quite a lot. On the other hand, we have Lambda, which is serverless and scales automatically with zero maintenance, it offers a better cost-performance trade-off for small medium workloads.

2 Conclusion

The main research question I covered in this was: Which AWS compute services are most suitable for running large-scale Word Frequency Analysis (WFA), and what are the trade-offs in terms of scalability and performance?

Based on the benchmarking, implementation, and observation that I did, we can conclude that Lambda is better for the small word frequency analysis. This is due to its automatic scaling, low maintenance, and its faster processing time. It is especially fast with dataset up to ~120MB, but it's also very good with larger datasets like ~480MB. But running the Lambda word frequency analysis on your local setup can be time consuming, since the execution time increases drastically when the file content increases.

Amazon EC2, as we can conclude is also suitable for smaller datasets But when you are scaling up the content, it will start to struggle with resource limits and manual configuration limits. Also, since I used the free version, the performance plateaued due to memory limitations of the t2.micro instance. This shows that scaling for larger instance types of heavy workload is needed.

2.2 Recommendations

After putting all the pros and cons against each other, I would recommend Lambda for small to medium word frequency analysis workloads. This is due to not only it's cost-effectiveness, but also due its simplicity.

For larger scaled word frequency analysis? I would most likely recommend alternatives like AWS batch or EMR. They offer more control and scalability, and provide sufficient memory and CPU recourses.

For future research I would like to explore the larger Lambda configuration, I could evaluate the cost-performance trade-offs over extended workloads by using the detailed billing data.

PoC (Appendix A)

```
nano wfa.py
echo "hello world hello aws cloud aws cloud hello" > dataset_small.txt
```

Figure 1, creating the dataset_small.txt with repeated words.

```
ubuntu@ip-172-31-38-108:~$ # Download ~20 public domain books from Project Gutenberg
for id in {1342,84,98,1661,2701,2542,11,1080,120,1232,5200,6130,174,98,12345,105,1184,74,36,43}; do
  wget https://www.gutenberg.org/files/$id/$id-0.txt -O book_$id.txt
done

# Combine into one medium file
cat book_*.txt > dataset_medium.txt
ls -lh dataset_medium.txt  # check file size
--2025-06-09 21:55:59--  https://www.gutenberg.org/files/1342/1342-0.txt
Resolving www.gutenberg.org (www.gutenberg.org)... 152.19.134.47, 2610:28:3090:3000:0:bad:cafe:47
Connecting to www.gutenberg.org (www.gutenberg.org)|152.19.134.47|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 752575 (735K) [text/plain]
Saving to: 'book_1342.txt'
```

Figure 2, downloading the e-books from Gutenberg and creating the dataset_medium.txt

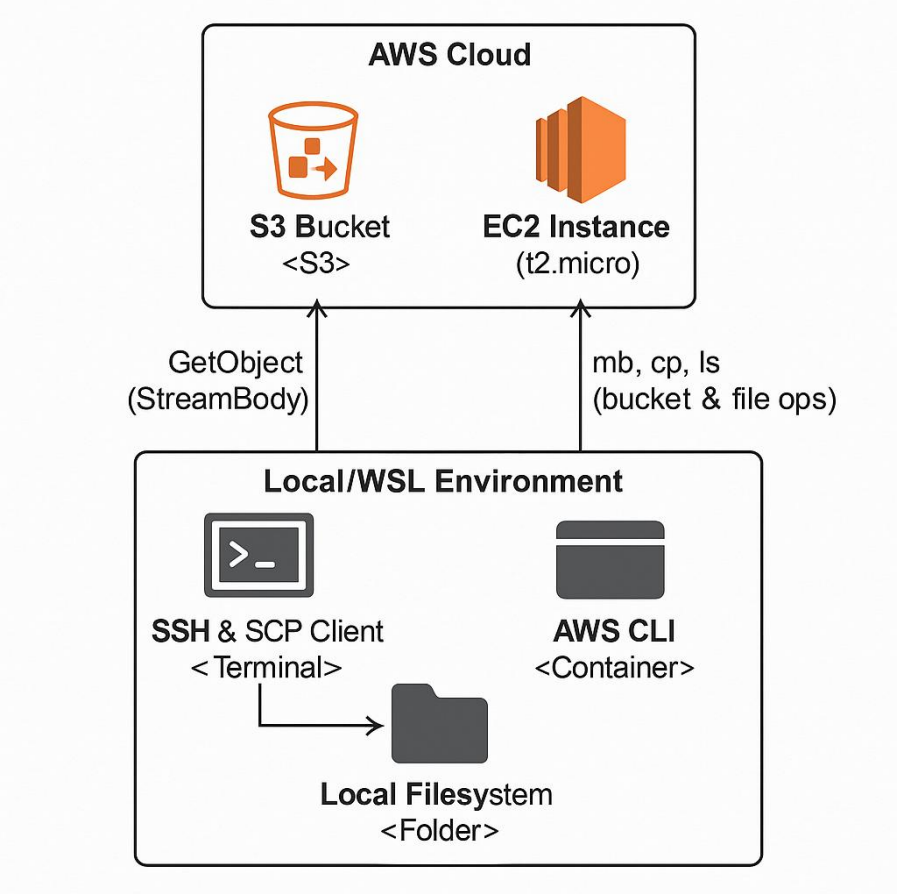

Figure 3, UML component diagram for EC2.

```
joeri@laptop-joeri:/mnt/c/Users/joeri/downloads$ ssh -i ~/keys/wfa-key.pem ubuntu@13.60.74.51
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.8.0-1029-aws x86_64)
```

Figure 4, Showing that EC2 is correctly installed with ubuntu and connected.

```python
from collections import Counter
import sys

def word_count(file_path):
    with open(file_path, 'r', encoding='utf-8') as f:
        words = f.read().lower().split()
        freq = Counter(words)
        for word, count in freq.most_common(10):
            print(f"{word}: {count}")

if __name__ == "__main__":
    word_count(sys.argv[1])
```

Figure 5, python code for checking WFA.

```
ubuntu@ip-172-31-38-108:~$ python3 wfa.py dataset_small.txt
hello: 3
aws: 2
cloud: 2
world: 1
```

Figure 6, WFA on small txt file containing the count of the words.

```
ubuntu@ip-172-31-38-108:~$ time python3 wfa.py dataset_medium.txt
the: 111576
and: 58059
of: 53512
to: 49638
a: 37172
in: 29476
i: 26213
his: 22289
he: 21835
that: 21215

real    0m0.637s
user    0m0.507s
sys     0m0.130s
ubuntu@ip-172-31-38-108:~$ time python3 wfa.py dataset_large.txt
Traceback (most recent call last):
  File "/home/ubuntu/wfa.py", line 12, in <module>
    word_count(sys.argv[1])
  File "/home/ubuntu/wfa.py", line 6, in word_count
    words = f.read().lower().split()
            ^^^^^^^^^^^^^^^^
MemoryError

real    0m1.019s
user    0m0.435s
sys     0m0.414s
ubuntu@ip-172-31-38-108:~$
```

Figure 7, showing the medium vs large dataset, the processing time, the word count, and the memoryError.

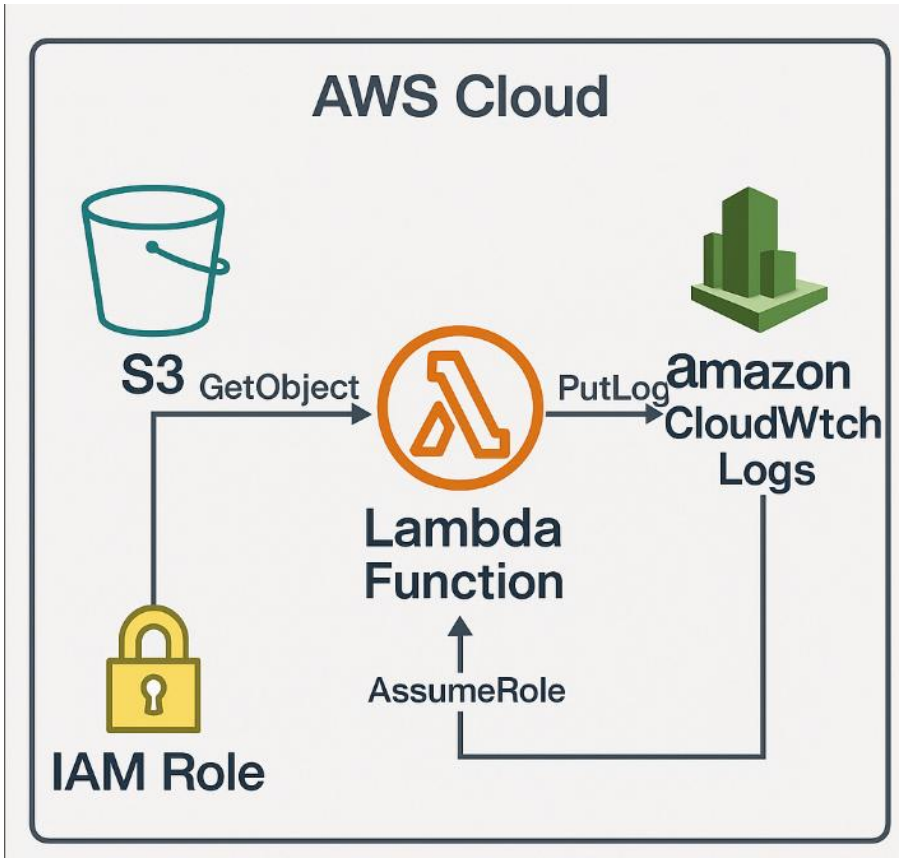

Figure 8, Lamda UML component diagram.

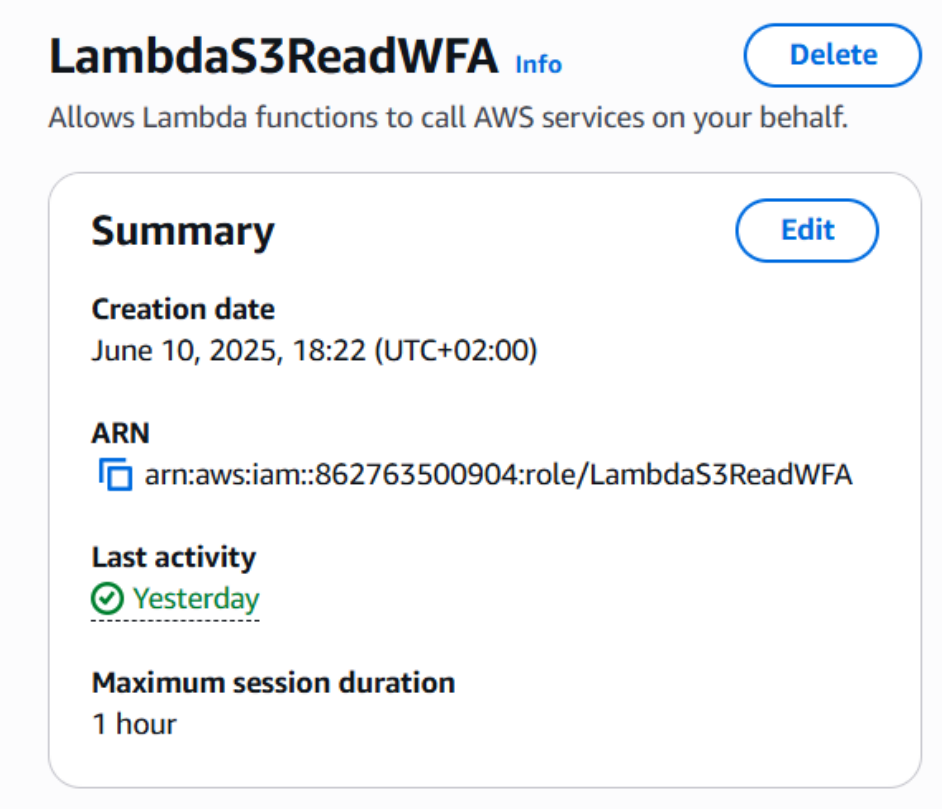

Figure 9, Creation of my IAM role.

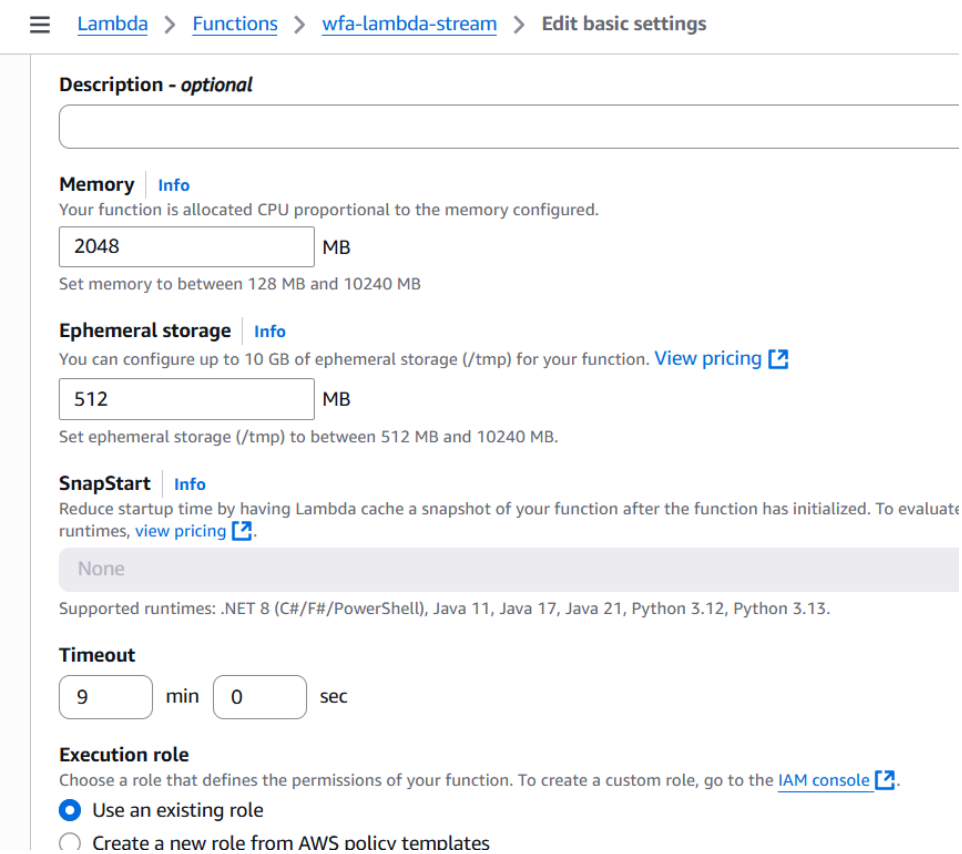

Figure 10, Showing the path to function and general configuration.

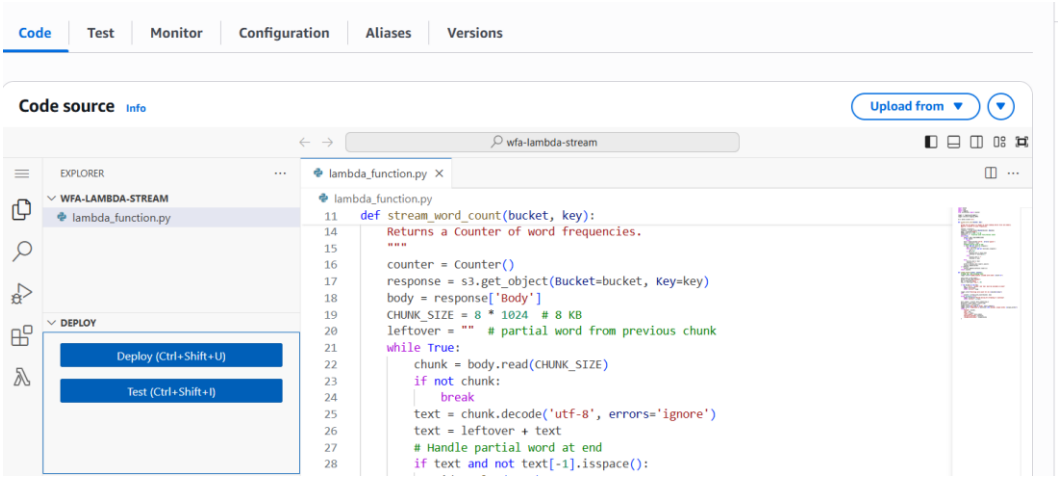

Figure 11, WFA code inside the handler.

Executing function: succeeded (logs)

Details


```
{
  "bucket": "wfa-bucket-joeri-202506",
  "key": "input/dataset_small.txt",
  "top_n": 10,
  "most_common": [
    [
      "hello",
      3
    ],
    [
```


Summary

Code SHA-256 /u86Tx4wmoxIlS+5gX76qYcwJIdAdmcOhegPemmi73E=	**Execution time** 10 seconds ago
Function version $LATEST	**Request ID** 631698c3-23dc-4db5-bdcf-42ec887b79d6
Duration 62.76 ms	**Billed duration** 63 ms
Resources configured 512 MB	**Max memory used** 83 MB

Log output

Figure 12, output from Lambda small dataset WFA.

Details


```
{
  "bucket": "wfa-bucket-joeri-202506",
  "key": "input/dataset_medium.txt",
  "top_n": 10,
  "most_common": [
    [
      "the",
      218547
    ],
    [
      "and"
```


Summary

Code SHA-256 u86Tx4wmoxIlS+5gX76qYcwJIdAdmcOhegPemmi73E=	**Execution time** 48 minutes ago
Function version LATEST	**Request ID** 7e3da01a-3491-4e7a-96c4-fe04f5dea5aa
Duration 2613.42 ms	**Billed duration** 2614 ms

Figure 13, WFA on the middle dataset.

Function version $LATEST	**Request ID** ef97f3ae-aaf8-48de-80d7-4a3f9d8117da
Duration 6393.60 ms	**Billed duration** 6394 ms
Resources configured 2048 MB	**Max memory used** 103 MB
Init duration 503.50 ms	

Log output

The area below shows the last 4 KB of the execution log. Click here to view the corresponding CloudWatch log group.


```
[INFO]  2025-06-10T17:45:41.931Z                Found credentials in environment variables.
START RequestId: ef97f3ae-aaf8-48de-80d7-4a3f9d8117da Version: $LATEST
[INFO]  2025-06-10T17:45:42.126Z        ef97f3ae-aaf8-48de-80d7-4a3f9d8117da    lambda_handler invoked with event: {'bucket': 'wfa-bucket-joeri-202506', 'key':
'input/dataset_large.txt', 'top_n': 10}
[INFO]  2025-06-10T17:45:42.126Z        ef97f3ae-aaf8-48de-80d7-4a3f9d8117da    Starting word count for s3://wfa-bucket-joeri-202506/input/dataset_large.txt
[INFO]  2025-06-10T17:45:48.510Z        ef97f3ae-aaf8-48de-80d7-4a3f9d8117da    Top 10 words: [('the', 874191), ('and', 474161), ('of', 428914), ('to', 416016),
('a', 305834), ('in', 234620), ('he', 192348), ('i', 191895), ('that', 182033), ('his', 176280)]
[INFO]  2025-06-10T17:45:48.510Z        ef97f3ae-aaf8-48de-80d7-4a3f9d8117da    Completed in 6.38 seconds; unique words: 170608
END RequestId: ef97f3ae-aaf8-48de-80d7-4a3f9d8117da
REPORT RequestId: ef97f3ae-aaf8-48de-80d7-4a3f9d8117da  Duration: 6393.60 ms    Billed Duration: 6394 ms        Memory Size: 2048 MB    Max Memory Used: 103 MB
```


Figure 14, WFA on large dataset.

References

Project Gutenberg. (n.d.). Project Gutenberg. https://www.gutenberg.org/